GDB

What is GDB?

- GDB: The GNU Project Debugger
- Allows you to see what is going on “inside” another program while it executes -- or what another program was doing at the moment it crashed.
- Step through lines of code, functions, etc
- View registers, memory
- Breakpoints

Why use GDB?

- Use GDB. It will tell you significantly more
  - Seg Fault/Trap is pretty useless
  - Invalid accesses on line number ## is more helpful
  - Printf won't be super useful when doing assembly
- GDB testimonials from previous students:
  - *"I couldn't have done this without that GDB! You were right. It is awesome!"*
  - *"I really learned how to use the GDB debugger. It is now my best friend, and I will love and cherish its beauty forever."*
- I cannot help you without specific information
- Stay after class/Bring your laptop

Compiler Support for Debuggers

- GNU Compiler Collection

```
gcc [flags] <source files>
ex: gcc hello.c  //produces a.out
```

- Useful Flags

Flag	Purpose
-v	Verbose output during build
-o <outFileName>	Save output to a specific file
-g	Build with debug symbols
-Werror	Make all warnings into errors
-m32	Build 32 bit code (default is 64 bit)

- Visual Studio: PDBs, Xcode -> .d files

Why not recent debuggers?

- Most Debuggers are based on GDB
- GDB has existed since 1986
- GDB (stubs) exist for many, many platforms
- Front-ends use it (Eclipse, EDB, VS, xCode)
- Commands are similar to other debuggers
- Concepts are similar
- GDB foo is impressive

Using GDB

- gdb programname.exe
- **SHOULD** compile with –g option (builds in debug information)
  - gcc –g –o hello hello.c

- GDB COMMANDS/tutorial
  - http://www.yolinux.com/TUTORIALS/GDB-Commands.html
  - http://www.cprogramming.com/gdb.html
- help – lists gdb command topics

GDB Commands Cont.

- run – starts execution
- start – start execution but pause on the first instruction
- quit – quit gdb, short cut is q
- s – step one instruction
- c – continue execution
- finish – continue until current function finishes
- next – step one instruction but do not enter functions

GDB Commands

- break *function* – break each time *function* is entered
- break # - break at line #
- break +*offset* – break *offset* lines forward from current line
- break – *offset* – break *offset* lines backward from current line
- break #/*funtion* if *condition* – break only if condition is met

GDB Commands Cont.

- info break – display information of all current breakpoints
- clear #/*function* – clear any breakpoints at # or *function*
- delete – delete all breakpoints/watchpoints
- enable *bp#* - enable breakpoint *bp#. bp#* is the index seen during ‘info break’ command
- disable *bp#* - disable breakpoint *bp#*. bp# is the index seen in ‘info break’ command

GDB Commands Cont.

- commands *bp#*
  - *commandlist* – commands to execute every time *bp#* is hit
  - *ex:* `commands 1`

    ```
    >print index
    >continue
    >end
    ```

    Prints the value of variable index then continues execution

- watch *variable* – break if *variable* is written to
- rwatch *variable* – break if *variable* is read from

GDB Example

```
>gcc -m32 -g myProgram.c -o
myProgram
>gdb myProgram
(gdb)start
(gdb)s
(gdb)c
(gdb)exit
```

start but pause on first instruction

execute 1 instruction

execute until end of program of hit a breakpoint

exit gdb

Building for GDB

- Typical gcc compilation
  - `gcc -m32 program.c -o program`
- If you want to debug with GDB, add the –g flag
  - `gcc -m32 -g program.c -o program`

GDB Example

```
#include <stdio.h>

long long factorial(int n);

int main(int argc, char** argv)
{
    int n;
    sscanf(argv[1],"%d",&n);
    printf("%lld\n",factorial(n));
    return 0;
}

long long factorial(int n)
{
    int i;
    long long f=1;
    for (i=1;i<=n;i++) {
        f*=i;
    }
    return f;
}
```

A simple program to step through

We want to focus on the factorial function

GDB Example

```
$ gdb ./factorial
(gdb) list
1   #include <stdio.h>
2
3   long long factorial(int n);
4
5   int main(int argc, char** argv)
6   {
7       int n;
8
9       sscanf(argv[1],"%d",&n);
10
(gdb) list factorial
12
13      return 0;
14  }
15
16  long long factorial(int n)
17  {
18      int i;
19      long long f=1;
20      for (i=1;i<=n;i++) {
21          f*=i;
```

I first list the source from the beginning , to figure out what I'm looking at. I see the factorial function declaration.

Knowing the function name, I can now list the source for that function.

GDB Example

```
(gdb) b factorial
Breakpoint 1 at 0x804846d: file factorial.c, line 19.
(gdb) start 10
Temporary breakpoint 2 at 0x804841d: file factorial.c, line 9.
Starting program: /home/deltaop/cse2421/example_programs/factorial/factorial 10

Temporary breakpoint 2, main (argc=2, argv=0xbffff3c4) at factorial.c:9
9        sscanf(argv[1],"%d",&n);
(gdb) continue
Continuing.

Breakpoint 1, factorial (n=10) at factorial.c:19
19       long long f=1;
(gdb) list
14   }
15
16   long long factorial(int n)
17   {
18       int i;
19       long long f=1;
20       for (i=1;i<=n;i++) {
21           f*=i;
22       }
23       return f;
```

I set a breakpoint on the factorial function

I start the program, passing in 10 as a command line parameter. The start command breaks on the first instruction, so I continue.

The factorial breakpoint is hit. I use list to get my bearings.

GDB Example

```
(gdb) step
20        for (i=1;i<=n;i++) {
(gdb) print f
$1 = 1
(gdb) display f
1: f = 1
(gdb) step 5
21            f*=i;
1: f = 2
(gdb) step 5
20        for (i=1;i<=n;i++) {
1: f = 120
```

I want to examine the factorial calculation (f), but it is not yet initialized, so I step one instruction.

After it is initialized, I can print f to examine its value.

I ‘display f’ to tell gdb to print the value of f automatically from now on

I then step through 10 lines of source (a few iterations of the factorial loop)

GDB Example

```
(gdb) list
15
16   long long factorial(int n)
17   {
18        int i;
19        long long f=1;
20        for (i=1;i<=n;i++) {
21             f*=i;
22        }
23        return f;
24   }
(gdb) b 23
Breakpoint 3 at 0x80484b9: file factorial.c, line 23.
(gdb) c
Continuing.

Breakpoint 3, factorial (n=10) at factorial.c:23
23        return f;
1: f = 3628800
```

I really want to see the value coming out of the function. I list the source again.

I set a breakpoint on the last line of the function

And continue execution; the program runs to the breakpoint. I see that the function is returning 3628800

GDB Example

```
(gdb) c
Continuing.
3628800

Program exited normally.
```

I've finished stepping through the factorial function, so I allow the program to continue execution. GDB shows me the program's output (3628800), and tells me the program has finished.

~/.gdbinit

Saves history across invocations of gdb:

set history filename ~/.gdb_history

CSE 2421

Colors!!!

basic ~/.vimrc

```
1 #include <stdio.h>
2 long long factorial(int n);
3 int main(int argc, char** argv)
4 {
5  int n; // I lawefaawefs comwefwaefewfment better
6  sscanf(argv[1],"%d",&n); // we should probably check to
  uit if the user does not pass something in
7  printf("%lld\n",factorial(n));
8  return 0; //this is another comment
9 }
10 long long factorial(int n)
11 {
12  int i;
13  long long f=1;
14  for (i=1;i<=n;i++) {
15  f*=i;
16  }
17  return f;
18 }
```

```
syntax on                "Colors!"
set number               "Line numbers!"
set nocompatible         "Make backspace work!"
set backspace=2

set tabstop=4            " The width of a TAB is set to 4.
                         " Still it is a \t. It is just that
                         " Vim will interpret it to be having
                         " a width of 4.

set shiftwidth=4         " Indents will have a width of 4
set softtabstop=4        " Sets the number of columns for a TAB
set expandtab            " Expand TABs to spaces
```

The Ultimate vimrc

https://github.com/amix/vimrc

```
git clone --depth=1
  https://github.com/amix/vimrc.git ~/.vim_runtime

sh ~/.vim_runtime/install_awesome_vimrc.sh
```